System Architecture — eRegistrar

Course: CS425 — Software Engineering
Student: Ziad El Fatih — 618971

First iteration of architectural analysis, based on the Vision Document and the SRS.

---

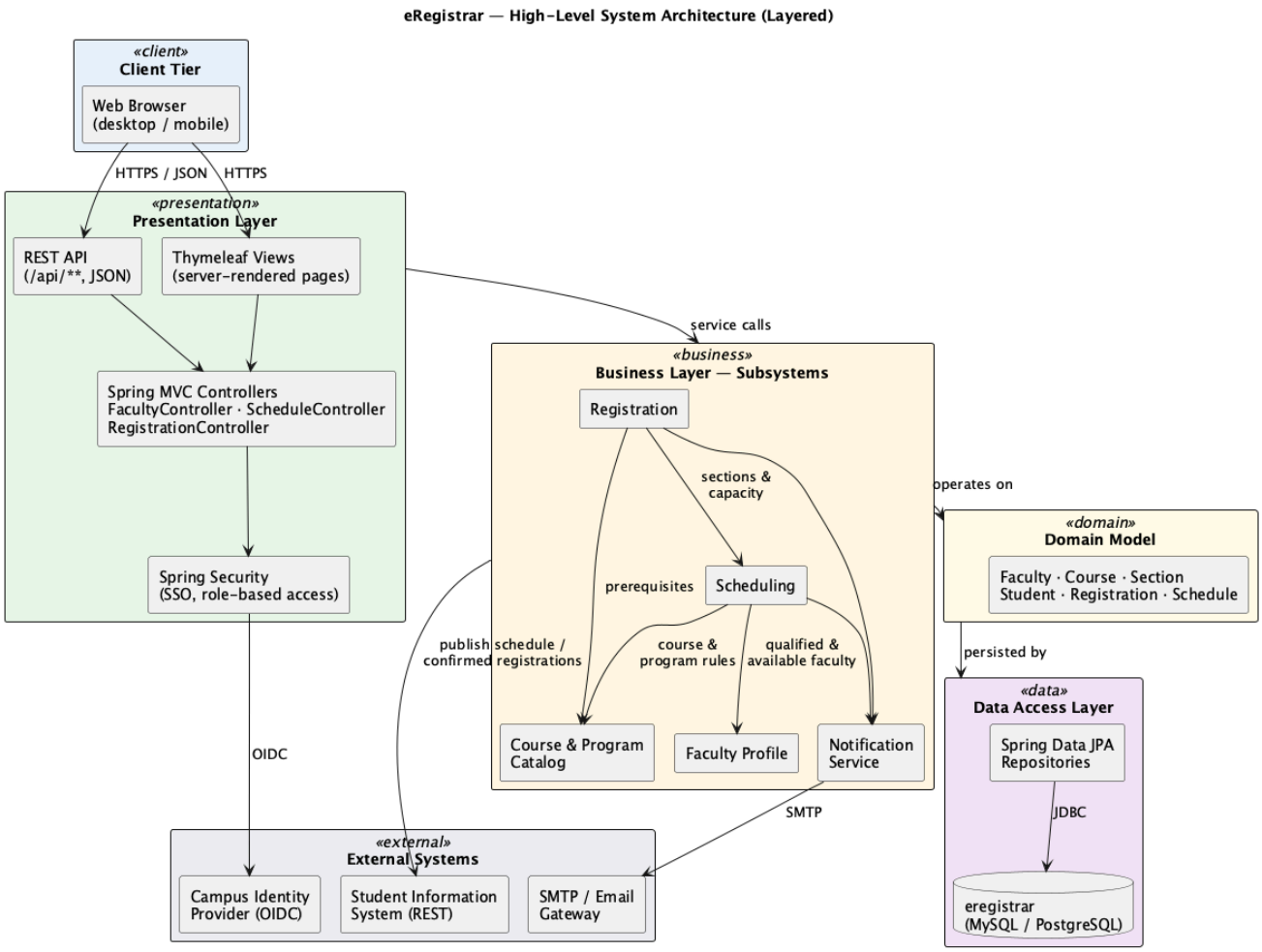
1. Constraints and Drivers

#	Constraint or quality attribute	Effect on the architecture
C1	Browser-only client; nothing installed on user machines	Server-side web application
C2	Java / Spring Boot technology stack	Layered Java web architecture
C3	Highly relational data (courses, prerequisites, sections, registrations)	Relational database with JPA
C4	Authentication delegated to the campus identity provider	Security handled at the presentation boundary
C5	Section capacity must hold under concurrent registration (NFR3)	Registration must be one server-side transaction with locking
C6	150 concurrent users at the registration peak (NFR2)	Stateless application tier
C7	Scheduling and registration rules are the volatile part of the system	Rules isolated in a business layer, away from the UI
C8	Small team, one term, modest hardware	A single deployable application, not distributed services

2. Architecture Selected

A **layered web architecture**: Presentation → Business (subsystems) → Domain Model → Data Access, with external systems behind adapters. Dependencies point downwards only.

Two alternatives were considered and rejected. A **monolithic single-layer** application would be quickest to build but would scatter the volatile scheduling and registration rules through the page code, failing C7. **Microservices** would allow independent scaling, but splitting registration from scheduling turns the capacity check into a distributed transaction — a direct threat to C5 — and the operational cost is not affordable under C8. The layered design keeps registration inside one local transaction while still drawing subsystem boundaries along which services could be extracted later.



3. Layers

Layer	Responsibility	Key elements
Presentation	HTTP handling, view rendering, authentication and role checks	Spring MVC controllers, Thymeleaf views, Spring Security
Business	All business rules and transaction boundaries, split into subsystems	Faculty Profile, Catalog, Scheduling, Registration, Notification
Domain model	The concepts and invariants of the problem, independent of frameworks	Faculty, Course, Section, Student, Registration, Schedule
Data access	Persistence and queries — the only layer that knows SQL	Spring Data JPA repositories, MySQL/PostgreSQL
External	Systems outside our control, reached through adapters	Campus identity provider, SIS, SMTP

4. Subsystems

Subsystem	Owns	Provides	Depends on
Faculty Profile	Profiles, specializations, teachable courses, availability	`FacultyService`	Domain, repositories
Course & Program Catalog	Courses, prerequisites, programs, blocks	`CatalogService`	Domain, repositories
Scheduling	Schedules, sections, faculty assignment, conflict detection	`ScheduleService`	Faculty Profile, Catalog
Registration	Registrations, rule validation, seat management	`RegistrationService`	Catalog, Scheduling
Notification	Notifying affected users of schedule changes	`NotificationService`	SMTP adapter

The dependency graph is acyclic — Registration → Scheduling → {Faculty Profile, Catalog} — which is what makes the subsystems separately testable.

5. Key Mechanisms

Mechanism	Approach
Persistence	JPA/Hibernate through Spring Data repositories
Transactions	Declarative (`@Transactional`) at the business-service boundary; one registration = one transaction
Concurrency	Optimistic locking (`@Version`) on `Section` ; the seat check and the seat decrement share a transaction, so capacity can never be exceeded (NFR3, BR7)
Security	Single sign-on against the campus identity provider; roles Admin/Faculty/Student; authorization re-checked in the business layer
Validation	Input format checked in the presentation layer; business rules enforced only in the business layer
External integration	Identity provider, SIS and SMTP behind adapter interfaces, stubbed during development

6. Risks

#	Risk	Mitigation
R1	Schedule generation gets harder as constraints multiply	Iteration 1 uses greedy assignment and reports unstaffed courses explicitly; measure before considering a constraint solver
R2	Contention on popular sections at the registration peak	Optimistic locking with short transactions; load-test the registration path before the first live window
R3	SIS integration details are unknown	Adapter interface with a stub, so the core flows are demonstrable without it